

NATIONAL RESEARCH UNIVERSITY
HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "Data Science and Business Analytics"

**Research Project Report on the Topic:
Lung Disease Detection**

Fulfilled by:

Student of the Group БПАД221
Zagoskin Egor Igorevich

27.05.2025

(date)

Assessed by the Project Supervisors:

Ryabtsev Dmitry Igorevich
Associate Professor
Faculty of Computer Science, HSE University

Vasiliev Boris Alekseevich
AI engineer
Project group "Information systems for medical applications"

Moscow 2025

Contents

Annotation	3
Аннотация	3
1 Introduction	4
1.1 Basic terms and definitions	4
1.2 Background	4
2 Literature Review	5
2.1 Dataset	5
2.2 Existing solutions	6
3 Comparison of models	7
4 Augmentations	9
4.1 Singular augmentatoin	9
4.2 Multiple augmentations	10
5 Freeze layers and final training	11
6 Knowledge distillation	12
7 Conclusion	12
References	13

Annotation

This project is aimed at improving the quality of existing ML models for detecting lung diseases. It tests the performance of a wide range of neural networks on this specific task and aims to surpass the quality of existing solutions.

Аннотация

Данный проект заключается в улучшении качества существующих ML моделей для детекции заболеваний легких. Он тестирует широкий спектр нейронных сетей для решения именно этой задачи и стремится превзойти качество существующих моделей.

Keywords

ML, CV, CNN, Bioinformatics, Classification, Segmentation, Transfer learning, Knowledge distillation

1 Introduction

1.1 Basic terms and definitions

Computer vision (CV) – a field of ML that focuses on enabling machines to interpret, analyze, and understand visual information from the world, such as images and videos.

Classification – a type of supervised learning where data is categorized into predefined classes or labels. It involves training a model on a labeled dataset, where the input data is associated with the correct output class. The goal is to use this trained model to predict the class of new, unseen data points.

Segmentation – a process of partitioning an image into multiple regions or segments that share similar characteristics. The goal is to simplify or analyze the image by isolating objects or regions of interest.

Neural network – a powerful ML model inspired by the structure and function of the human brain. Neural networks consist of interconnected layers of nodes, called neurons, which work together to process input data and produce output predictions.

Convolutional neural network (CNN) – a type of neural networks based on the idea of consequently applying filters and combining their results in order to extract key features of input data. CNNs are able to take into account spatial structure of data and are mostly used for image processing.

Transfer learning – a machine learning technique where a model developed for one task is reused as the starting point for a model on a second, related task. Instead of training a model from scratch, transfer learning leverages knowledge (e.g., learned features, weights, or representations) from a pre-trained model, which was originally trained on another dataset, and adapts it to a new, often smaller, dataset.

Knowledge distillation – a process of training a smaller model utilizing a pretrained large model. The resulting model is supposed to perform better than if the larger model was not used.

1.2 Background

In the last decade Artificial Intelligence proved itself to be applicable in a wide range of tasks. For instance, its subfields, computer vision, is now used in industrial robots, surveillance systems, autonomous vehicles and others. One of the most promising applications of CV is processing of medical images of various nature (e.g. simple photographs, X-ray scans, ultrasonic images). Since this field is extremely young, but very broad, there exist a great number of problems which were not properly addressed yet and many tasks only have open-source solutions of quite poor quality.

To diagnose a disease is a hard task which can now be effectively performed only by highly educated medical experts. Considering that the vast majority of the world population live in developing countries, there are billions of those who cannot get proper diagnoses [1]. And even for those who can it sometimes may cost a fortune. AI models for disease detection could mitigate this problem by making diagnosing more accessible and affordable [2, 3].

This work is focused on improving the quality of existing solutions for detecting COVID-19 and viral pneumonia from X-ray images. These diseases are extremely widespread and kill millions of people every year [4, 5]. A good-quality ML model could significantly improve the quality of early diagnosis and at the same time reduce the burden on in-demand medical experts, thus saving an immense number of lives.

2 Literature Review

2.1 Dataset

To have an easy access for alternative solutions, which can be used as benchmarks, I decided to go with one of the popular Kaggle datasets. Here are some of the gold medal datasets with above 8.00 usability which I was choosing from:

- 1) COVID-19 Radiography Database [6] with 1000+ upvotes and 400+ notebooks. It consists of 21165 images of classes Normal, Lung Opacity, Viral Pneumonia, and COVID.
- 2) CoronaHack -Chest X-Ray-Dataset [7] with 500+ upvotes and 100+ notebooks. This dataset contains 5933 images classified as Normal or Pneumonia.
- 3) Chest CT-Scan images Dataset [8] with 350+ upvotes and 100+ notebooks. It contains 1000 images classified as Normal or one of the three types of cancer (Adenocarcinoma, Large cell carcinoma, Squamous cell carcinoma).
- 4) Covid-19 Image Dataset [9] with 250+ upvotes and 150+ notebooks. This dataset contains 317 images which are Normal, Viral Pneumonia or Covid.
- 5) COVIDx CXR-4 [10] with 200+ upvotes and 39 notebooks. It consists of 84818 images classified as COVID-19 Negative or COVID-19 Positive.

Datasets 3 and 4 contain far too few images. Dataset 5 has a very modest number of existing solutions, and because of it even if I build a model which outperforms them all, it will be unclear whether it is because my model is good, or because very few people worked with this specific dataset. As for the dataset 2, despite seeming to be quite suitable, it loses to the dataset 1 in both popularity and amount of data. Moreover, dataset 1 is the only one that also contain lung masks which might

be helpful to further improve the quality of a solution.

So, the best choice is COVID-19 Radiography Database, a dataset of chest X-ray images ([Figure 1](#)) of four categories, which are COVID, Viral Pneumonia, Lung Opacity and Normal. The dataset contains 21,165 grayscale images of size 3 x 299 x 299 and corresponding lung masks.

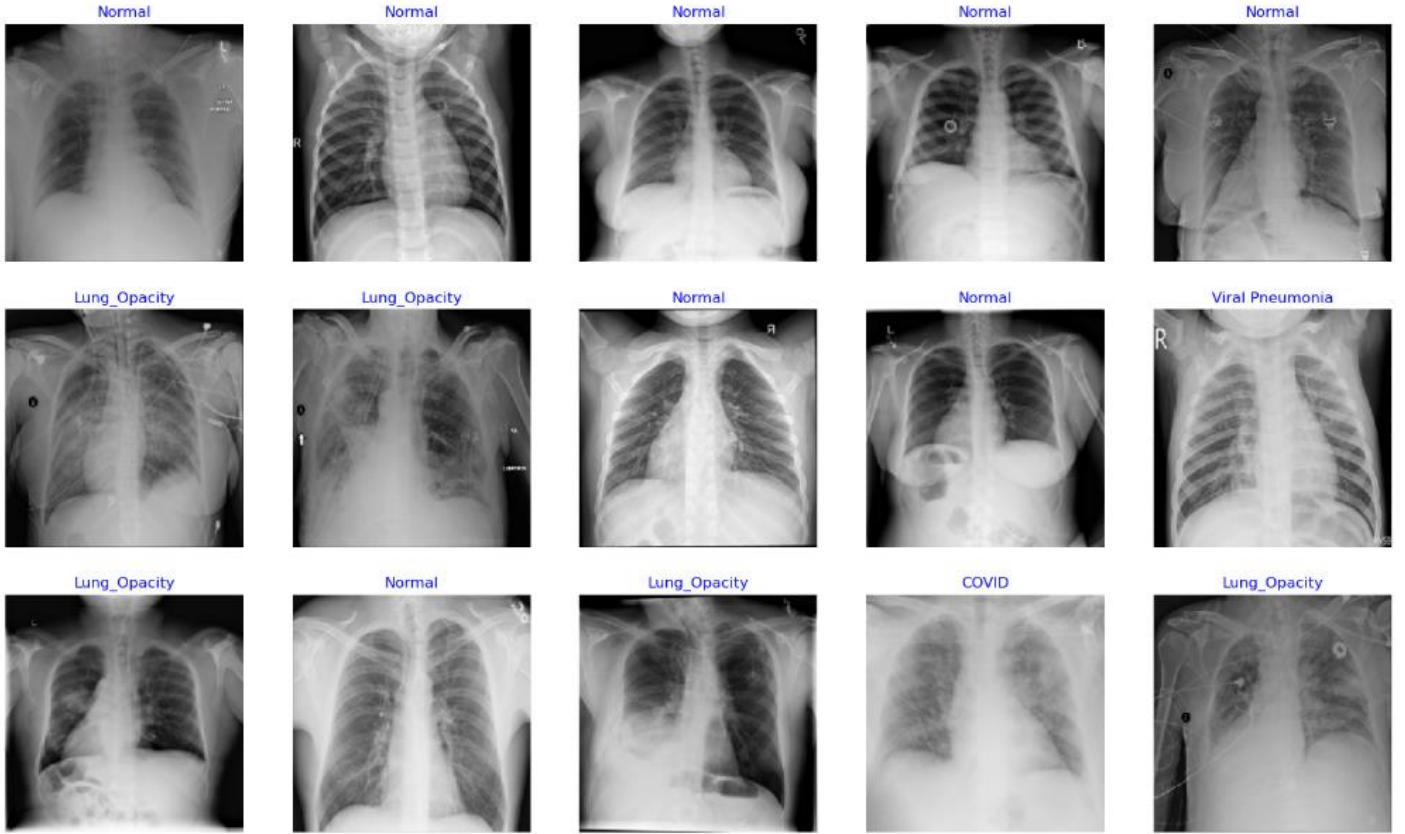


Figure 1 – Examples of chest x-ray images.

2.2 Existing solutions

As base solutions I chose the six that have received the most upvotes recently and of all time. The description of these solutions is below and in [Table 1](#). It can be seen that EfficientNets are showing very impressive results being both smaller and more accurate than older, more traditional models.

- 1) Solution [[11](#)] by Duy Trung Nguyễn using an Xception-based model combined with some image augmentations (rescale, rotate, sheer/zoom, flip). The model achieves an accuracy of 80.26%.
- 2) Solution [[12](#)] by Ahmed Hafez using an VGG19-based model. The model achieves an accuracy of 94.85%.
- 3) Solution [[13](#)] by Abdallah Wagih Ibrahim using an EfficientNetB0-based model combined with some image augmentations (rescale, rotate, flip). The model achieves an

accuracy of 96.32%.

- 4) Solution [14] by Chrysovalantis Constantinou using a model based on DenseNet201, ResNet50, VGG16 and XGBoost combined with some image augmentations (rescale, rotate, flip, affine, color). The model achieves an accuracy of 83.84%.
- 5) Solution [15] by Yossef Mohammed using an EfficientNetB1-based model combined with some image augmentations (flip). The model achieves an accuracy of 95.7%.
- 6) Solution [16] by Nam Pham 79 using an EfficientNetB0-based model combined with some image augmentations (flip). The model achieves an accuracy of 95.9%.

Table 1 – Comparison of the existing solutions.

#	Base model	Number of parameters	Accuracy
1	Xception	22M	80.26%
2	VGG19	20M	94.85%
3	EfficientNetB0	4.4M	96.32%
4	DenseNet201, ResNet50, VGG16, XGBoost	62M	83.84%
5	EfficientNetB1	6.7M	95.7%
6	EfficientNetB0	4.4M	95.9%

3 Comparison of models

To train models I used Nvidia Tesla T4 16Gb GPU provided by Google Colab and framework PyTorch. I divided the dataset into training, validation, and test parts in a proportion of 70 : 15 : 15 in a stratified manner and proceeded to testing a performance of a range of models pretrained on ImageNet-1K. For this initial comparison all models were trained for 5 epochs with validation after each epoch and the best validation accuracies out of 5 were chosen as the results of the models. I used cross-entropy as a loss function, AdamW with learning rate 0.001 as an optimizer, batch size of 16 images, and no image augmentation aside from normalization. The experiment was repeated 3 times for each model, average best scores (validation) are presented in [Table 2](#).

In order to speed up the training the 3 datasets of PIL images (train, val, test) were transformed to 3 torch tensors beforehand (in fact, this increased the speed of training several times, since before this the loading speed was a huge bottleneck). Also, to further increase training speed and save memory, I used float16 format instead of the usual float32.

Table 2 – Choice of the base model.

Model	Number of parameters	Years of publication	Accuracy
vgg19_bn	26M	2014-2015	89.57%
resnet50	24M	2015	90.89%
inception_v3	25M	2015	91.22%
wide_resnet50_2	67M	2016-2017	89.27%
resnext50_32x4d	23M	2016-2017	92.06%
densenet201	18M	2016-2018	92.85%
mobilenet_v3_large	3M	2019	93.21%
regnet_y_1_6gf	10M	2020	92.56%
regnet_y_3_2gf	18M	2020	94.02%
regnet_y_8gf	37M	2020	91.36%
efficientnet_v2_s	20M	2021	93.81%
swin_v2_t	28M	2021-2022	92.17%
convnext_tiny	28M	2022	92.74%
maxvit_t	30M	2022	89.86%

The best performing model was regnet_y_3_2gf with modest 18 million parameters and accuracy above 94%. After selecting regnet_y_3_2gf as the base model I tried to improve the results using the segmentation masks provided with the dataset.

Method 1) I multiplied images with the corresponding segmentation masks, thus, nullifying all the pixels which do not belong to the lung masks. The modified inputs were then used to train regnet_y_3_2gf network in the same way as described above.

Method 2) I used the original images as the first channel, the modified images from the method 1 as the second channel, and the masks as the third channel for the method 2 inputs. The method of training is the same.

Unfortunately, the use of the segmentation masks did not improve the accuracy of the model. Hence, this path was abandoned since it would probably be not the most fruitful one given that even the provided masks did not bring positive results. The results of the methods 1 and 2 against the no segmentation method are presented in Table 3.

Table 3 – Training with segmentation masks.

Model	Accuracy
No segmentation	94.02%
Method 1	84.46%
Method 2	92.17%

4 Augmentations

4.1 Singular augmentatoinis

The next step was to research effect of different augmentations. I began with image size augmentations such as CenterCrop and Resize to the sizes 272 x 272, 256 x 256, 224 x 224 and 192 x 192. Validation images were cropped / resized to the corresponding sizes. The best results for the both augmentations were shown by changing size to 256 x 256 with CenterCrop being slightly more effective. After discovering the most appropriate size, I also tried RandomResizedCrop but it did not improve the result. The comparison between size augmentations are presented in Table 4.

Table 4 – size augmentations.

Augmentation	Accuracy
No augmentation	94.02%
CenterCrop to 272	93.69%
CenterCrop to 256	94.53%
CenterCrop to 224	92.84%
CenterCrop to 192	93.50%
Resize to 272	93.56%
Resize to 256	94.28%
Resize to 224	92.33%
Resize to 192	94.18%
RandomResizedCrop (size=256, scale=(0.6, 1))	93.90%

Then I tried augmenting brightness, contrast and saturation with ColorJitter, sharpness with RandomAdjustSharpness, and contract with RandomAutocontrast, as well as using RandomHorizontalFlip. The best improvement in accuracy was achieved with RandomAdjustSharpness. ColorJitter and RandomHorizontalFlip also brought some improvements while RandomAutocontrast did not. The results are in Table 5.

Table 5 – other augmentations.

Augmentation	Accuracy
No augmentation	94.02%
ColorJitter (brightness=0.4, contrast=0.4, saturation=0.4)	92.70%
ColorJitter (brightness=0.3, contrast=0.3, saturation=0.3)	94.72%
ColorJitter (brightness=0.2, contrast=0.2, saturation=0.2)	93.66%
ColorJitter (contrast=0.3, saturation=0.3)	94.16%
ColorJitter (brightness=0.3, saturation=0.3)	92.43%
ColorJitter (brightness=0.3, contrast=0.3)	94.22%
RandomAdjustSharpness (sharpness_factor=2)	93.37%
RandomAdjustSharpness (sharpness_factor=1.7)	94.96%
RandomAdjustSharpness (sharpness_factor=1.5)	95.65%
RandomAdjustSharpness (sharpness_factor=1.3)	94.16%
RandomAutocontrast	92.39%
RandomHorizontalFlip	94.96%

4.2 Multiple augmentations

After researching the most successful augmentation (CenterCrop (size=256), ColorJitter (brightness=0.3, contrast=0.3, saturation=0.3), RandomAdjustSharpness (sharpness_factor=1.5) and RandomHorizontalFlip), I proceeded to testing their combinations. First, I tested all combinations of 4 or 3 of them and, after learning that the best combination was CenterCrop + RandomAdjustSharpness + RandomHorizontalFlip, I tested combinations of 2 out of these 3. From now on for all experiments the model is trained for 7 epochs instead of 5 as it was previously. The results are in Table 6.

Table 6 – multiple augmentations

Augmentation	Accuracy
No augmentation	94.02%
All 4 at once	93.36%
3 without Resize	93.87%
3 without RandomHorizontalFlip	94.70%
3 without ColorJitter	96.03%
3 without RandomAdjustSharpness	93.15%
Resize + RandomAdjustSharpness	93.94%
RandomHorizontalFlip + RandomAdjustSharpness	95.72%
Resize + RandomHorizontalFlip	93.74%

Overall, the best combinations of augmentations has given to the model additional 2% in accuracy.

5 Freeze layers and final training

The next step was to disable gradients for some initial layers in order to not interfere with capturing generic low-level features and preserve the most reusable knowledge from the training on Imagenet. Table 7 shows the results of the model depending on the number of frozen parameters (the lowest layers are frozen first) out of 18 million. The best result was achieved when freezing 3.5 million parameters which will be used from now on.

Table 7 – freezing layers.

Number of frozen parameters	Accuracy
0	96.03%
57,000	96.70%
850,000	96.16%
3,500,000	97.14%
6,400,000	96.29%
9,300,000	95.86%
13,100,000	95.30%

At the next step I trained a model using all of the mentioned above (regnet_y_3_2gf with CenterCrop (size256), RandomAdjustSharpness (sharpness_factor=1.5), RandomHorizontalFlip, freezing of 3.5 million parameters, cross-entropy as a loss function, AdamW with learning rate 0.001 as an optimizer) for 15 epochs with batch size 32 in order to determine the optimal number of epochs – 7. After that the final model was obtained via training for 11 epochs with MultiStepLR scheduler which decreased the learning rate tenfold after 7th and 9th steps. The model showed 97.16% accuracy on testing data.

6 Knowledge distillation

Finally, I conducted knowledge distillation from the described above model (teacher) into a student model based on mobilenet_v3_large (3m parameters). For this I adjusted the loss function so that it was based not only on the difference between predictions and true labels, but also on the difference between logits of the student and the teacher models, which allows the smaller model to “learn” from the larger and achieve higher results. The knowledge distillation was, first, conducted for 12 epochs to determine the optimal number of epochs – 6. Then, the final small model was obtained through training for 9 epochs with MultiStepLR scheduler which decreased the learning rate tenfold after 6th and 9th steps. The model showed 96.24% accuracy on testing data.

7 Conclusion

In this project I have researched suitability of a wide range of neural networks for the task of lung disease detection (surprisingly, the best model was neither one of the most famous, nor one of the newest) and managed to construct a model outperforming the best existing solution while trained on a less amount of data (80% of the dataset for the best existing solution and 70% for my solution). Also, I constructed a model which achieves the same accuracy as the best existing solution while being both smaller and using less data. Moreover, my work provides an example of using Pytorch which is very rare for the dataset in consideration (e.g. out of the 6 most popular solutions 5 use Keras and only one uses Pytorch).

Possibilities for the future work include: use of the images from several different positions as it is done by doctors, interpretation of the models, expansion of the solution to consider more variants of lung diseases.

References

1. World Health Organization. “World Bank and WHO: Half the world lacks access to essential health services, 100 million still pushed into extreme poverty because of health expenses”. URL: <https://www.who.int/news/item/13-12-2017-world-bank-and-who-half-the-world-lacks-access-to-essential-health-services-100-million-still-pushed-into-extreme-poverty-because-of-health-expenses> (date of request: 29.01.2025).
2. Weronika Dorocka. World Economic Forum. “How AI is improving diagnostics and health outcomes, transforming healthcare”. URL: <https://www.weforum.org/stories/2024/09/ai-diagnostics-health-outcomes> (date of request: 29.01.2025).
3. Saemoon Yoon, Amara Amadiogwu. World Economic Forum. “Emerging tech, like AI, is poised to make healthcare more accurate, accessible and sustainable”. URL: <https://www.weforum.org/stories/2023/06/emerging-tech-like-ai-are-poised-to-make-healthcare-more-accurate-accessible-and-sustainable> (date of request: 29.01.2025).
4. Ruuskanen O, Lahti E, Jennings LC, Murdoch DR. Viral pneumonia. *Lancet*. 2011 Apr 9;377(9773):1264-75. doi: 10.1016/S0140-6736(10)61459-6. Epub 2011 Mar 22. PMID: 21435708; PMCID: PMC7138033.
5. World Health Organization. “The top 10 causes of death”. URL: <https://www.who.int/news-room/fact-sheets/detail/the-top-10-causes-of-death> (date of request: 29.01.2025).
6. Amith Khandakar, Dr. Muhammad Chowdhury, Tawsifur Rahman. “COVID-19 Radiography Database”. URL: <https://www.kaggle.com/datasets/tawsifurrahman/covid19-radiography-database> (date of request: 29.01.2025).
7. Praveen. “CoronaHack -Chest X-Ray-Dataset”. URL: <https://www.kaggle.com/datasets/praveengovi/coronahack-chest-xraydataset> (date of request: 29.01.2025).
8. Mohamed Hany. “Chest CT-Scan images Dataset”. URL: <https://www.kaggle.com/datasets/mohamedhanyyy/chest-ctscan-images> (date of request: 29.01.2025).
9. Pranav Raikote. “Covid-19 Image Dataset”. URL: <https://www.kaggle.com/datasets/pranavraikokte/covid19-image-dataset> (date of request: 29.01.2025).

10. Alexander Wong, Andy, Hayden Gunraj, Hossein, May Pal, Naomi Waterloo.
“COVIDx CXR-4”. URL: <https://www.kaggle.com/datasets/andyczhao/covidx-cxr2>
(date of request: 29.01.2025).
11. Duy Trung Nguyễn. “Xception”. URL:
<https://www.kaggle.com/code/duytrungnguyn/xception> (date of request: 29.01.2025).
12. Ahmed Hafez. “Covid-19 Radiology | VGG19| f1-score: 95%”. URL:
<https://www.kaggle.com/code/ahmedtronic/covid-19-radiology-vgg19-f1-score-95>
(date of request: 29.01.2025).
13. Abdallah Wagih Ibrahim. “Covid-19 Classification | EfficientNetB0 | 96.32%”. URL:
<https://www.kaggle.com/code/abdallahwagih/covid-19-classification-efficientnetb0-96-32> (date of request: 29.01.2025).
14. Chrysovalantis Constantinou. “covid19_classification_xgboost_transfer_learning”.
URL:
<https://www.kaggle.com/code/cconsta1/covid19-classification-xgboost-transfer-learning> (date of request: 29.01.2025).
15. Yossef Mohammed. “covid19-detection|EfficientNetB1”. URL:
<https://www.kaggle.com/code/yossefmohammed/covid19-detection-efficientnetb1> (date of request: 29.01.2025).
16. Nam Pham 79. “Covid-19 Classification - DAT301m - NamPNH”. URL:
<https://www.kaggle.com/code/nampham79/covid-19-classification-dat301m-nampnh>
(date of request: 29.01.2025).